

Optimización heurística basada en búsquedas locales

Evaluación y comparación de algoritmos

Optimización Heurística y Aplicaciones

Máster Universitario en Ingeniería de Sistemas y Control



Curso 25/26

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

La evaluación y comparación de algoritmos de optimización permiten determinar la eficacia de un algoritmo y su rendimiento relativo frente a otros. Ambos procesos se basan en el uso de métricas específicas que proporcionan una visión cuantitativa del comportamiento del algoritmo.

- **Evaluación del rendimiento:** se centra en medir cómo de bien se comporta un algoritmo en términos de criterios específicos, como la calidad de la solución, la eficiencia computacional y la robustez. Las métricas utilizadas en la evaluación proporcionan una medida objetiva del éxito del algoritmo en alcanzar sus objetivos.
- **Comparación de algoritmos:** implica analizar el rendimiento de un algoritmo en relación con otros, utilizando las mismas métricas. Esto permite identificar cuál es más adecuado para un problema dado, considerando factores como la calidad de la solución y el tiempo de ejecución.

Beneficios de utilizar métricas comunes para evaluación y comparación:

- **Consistencia en el análisis:** al aplicar las mismas métricas, se asegura una evaluación coherente y comparable entre diferentes algoritmos.
 - **Identificación de fortalezas y debilidades:** permite detectar en qué aspectos un algoritmo supera a otros, facilitando la selección del más adecuado para un problema específico.
 - **Fomento de la mejora continua:** al comparar algoritmos, se pueden identificar áreas de mejora, impulsando el desarrollo de nuevas técnicas y enfoques.
- Elegir métricas relevantes para el tipo de problema y los objetivos del algoritmo, ya sean mono-objetivo o multi-objetivo.
 - Asegurar que las evaluaciones y comparaciones se realicen bajo condiciones reproducibles para garantizar la validez de los resultados.
 - Utilizar métodos estadísticos para validar los resultados y asegurar que las diferencias observadas no son producto del azar.

Al realizar una evaluación o comparación de algoritmos mono-objetivo, debemos tener presentes los siguientes aspectos:

- El algoritmo se centra en optimizar una **única función objetivo**, $f(x)$, que representa el rendimiento del algoritmo en términos de un solo criterio, como minimizar el coste o maximizar la eficiencia.
- Se pueden considerar varias **métricas de evaluación y comparación** del algoritmo:
 - **Calidad de la solución**: se mide directamente por el valor de $f(x)$.
 - **Tiempo de ejecución**: tiempo necesario para alcanzar la solución óptima o una solución aceptable.
 - **Número de iteraciones**: cantidad de iteraciones requeridas para converger a la solución.

Mientras que al realizar una evaluación de algoritmos multi-objetivo, tenemos:

- El algoritmo trabaja con **varias funciones objetivo**, $f_1(x), f_2(x), \dots, f_m(x)$, que deben ser optimizadas simultáneamente.
- Las **métricas de evaluación y comparación** son variadas, y giran en torno a los siguientes conceptos:
 - **Cobertura del frente de Pareto**: medida de cuán bien el algoritmo explora el espacio de soluciones.
 - **Diversidad**: evaluación de la distribución de las soluciones a lo largo del frente de Pareto.
 - **Convergencia**: proximidad de las soluciones al frente de Pareto ideal.

- En problemas multi-objetivo, la comparación es más compleja debido a la necesidad de considerar múltiples criterios simultáneamente. Se utilizan métricas como el hipervolumen y el indicador epsilon para comparar frentes de Pareto.
- Como herramientas de visualización, en multi-objetivo se utilizan además gráficos de frentes de Pareto y diagramas de dispersión para visualizar la diversidad y convergencia.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - **Métricas para algoritmos mono-objetivo**
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

La **función objetivo**, $f(x)$, es la métrica principal para evaluar y comparar la calidad de una solución en problemas de optimización mono-objetivo.

- El objetivo es encontrar una solución x^* tal que $f(x^*)$ sea mínimo (o máximo, dependiendo del problema).
- La calidad de un algoritmo se mide por su capacidad para aproximarse al valor óptimo $f(x^*)$.

El **tiempo de ejecución** es una métrica crítica que mide la eficiencia computacional de un algoritmo.

- Se define como el tiempo total requerido para que el algoritmo converja a una solución aceptable.
- Es importante considerar el tiempo de ejecución en relación con la calidad de la solución obtenida.

El **número de iteraciones** se refiere a la cantidad de pasos que un algoritmo realiza antes de converger a una solución.

- Un menor número de iteraciones puede indicar una mayor eficiencia del algoritmo, siempre que la calidad de la solución sea adecuada.
 - Es importante equilibrar el número de iteraciones con el tiempo de ejecución y la calidad de la solución.
- Al evaluar y comparar algoritmos, considerar el trade-off entre la calidad de la solución, el tiempo de ejecución y el número de iteraciones.
 - Un algoritmo que encuentra soluciones de alta calidad rápidamente y con pocas iteraciones es generalmente preferido.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - **Métricas para algoritmos multi-objetivo**
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Dominancia de Pareto: una solución x se dice que domina a otra solución y si se cumplen las siguientes condiciones:

- 1 $f_i(x) \leq f_i(y)$ para todos los objetivos i .
- 2 Existe al menos un objetivo j tal que $f_j(x) < f_j(y)$.

Interpretación:

- La solución x es al menos tan buena como y en todos los objetivos y estrictamente mejor en al menos uno.
- La dominancia de Pareto permite comparar soluciones considerando múltiples criterios simultáneamente.

El **frente de Pareto** es el conjunto de todas las soluciones no dominadas en el espacio de búsqueda. Formalmente, una solución x es parte del frente de Pareto si no existe otra solución y tal que y domine a x .

- **Importancia del frente de Pareto:**

- Representa el mejor compromiso posible entre los diferentes objetivos.
- Proporciona a los decisores un conjunto de soluciones óptimas entre las cuales elegir, según sus preferencias específicas.

- **Visualización:**

- En problemas con dos objetivos, el frente de Pareto se puede visualizar como una curva en un gráfico de dispersión.
- En problemas con más de dos objetivos, se utilizan técnicas de visualización más avanzadas para representar el frente.

El **hipervolumen** es una medida que cuantifica el volumen del espacio objetivo dominado por un frente de Pareto en relación con un punto de referencia.

- **Importancia:** un mayor hipervolumen indica que el frente de Pareto cubre una mayor parte del espacio objetivo, sugiriendo una mejor exploración y calidad de las soluciones.
- **Cálculo:** se calcula integrando el volumen del espacio entre el frente de Pareto y un punto de referencia, generalmente un punto peor que todas las soluciones del frente.

El **indicador epsilon** mide la mínima distancia multiplicativa necesaria para que un frente de Pareto sea al menos tan bueno como otro frente de referencia.

- **Importancia:** un menor valor del indicador epsilon denota que el frente de Pareto está más cerca del frente de referencia, sugiriendo una mejor calidad de las soluciones.
- **Cálculo:** se determina encontrando el menor valor de ϵ tal que para cada solución x en el frente de referencia, existe una solución y en el frente evaluado que cumple $f_i(y) \leq \epsilon \cdot f_i(x)$ para todos los objetivos i .

- Ambos indicadores permiten evaluar y comparar frentes de Pareto, proporcionando un análisis cuantitativo de su rendimiento.
- Utilizar gráficos de dispersión y diagramas de hipervolumen para ilustrar las diferencias en la cobertura y calidad de los frentes de Pareto.
- La elección del punto de referencia en el hipervolumen y la interpretación del indicador epsilon pueden influir en los resultados, por lo que es importante seleccionar estos parámetros cuidadosamente.

Además del **tiempo de ejecución** y el **número de iteraciones** ya vistos para algoritmos mono-objetivo, se pueden usar el **hipervolumen** o el **indicador epsilon** (con respecto a un punto o un frente de referencia, respectivamente) para evaluar y comparar algoritmos multi-objetivo.

Además, se pueden usar las siguientes métricas adicionales:

La **cobertura del frente de Pareto** mide la proporción o porcentaje del espacio objetivo que es dominado por el frente de Pareto obtenido por un algoritmo.

- **Importancia:** una mayor cobertura indica que el algoritmo ha explorado eficientemente el espacio de soluciones, encontrando un conjunto de soluciones que representan un buen compromiso entre los objetivos.
- **Cálculo:** se puede calcular como la proporción del hipervolumen del espacio objetivo cubierto por el frente de Pareto.

La **diversidad** evalúa la distribución de las soluciones a lo largo del frente de Pareto.

- **Importancia:** un frente de Pareto diverso proporciona a los decisores una amplia gama de soluciones entre las cuales elegir, aumentando la flexibilidad en la toma de decisiones.
- **Cálculo:** se pueden utilizar métricas como la distancia de dispersión, que mide la uniformidad de las soluciones a lo largo del frente. Se busca un valor bajo (próximo a 0).

La **convergencia** mide la proximidad de las soluciones obtenidas al frente de Pareto ideal.

- **Importancia:** una buena convergencia indica que el algoritmo ha encontrado soluciones cercanas al óptimo teórico, mejorando la calidad de las decisiones.
- **Cálculo:** se puede evaluar mediante la distancia promedio de las soluciones al frente de Pareto ideal, utilizando métricas como el indicador epsilon. Mejor cuanto más bajo.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - **Resumen de métricas**
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Algoritmos mono-objetivo

- **Calidad de la solución:** evaluada directamente a través del valor de la función objetivo $f(x)$.
- **Tiempo de ejecución:** medida de la eficiencia computacional necesaria para alcanzar la convergencia.
- **Número de iteraciones:** cantidad de pasos requeridos para obtener la solución.

Algoritmos multi-objetivo

- **Hipervolumen:** volumen del espacio dominado en relación con un punto de referencia.
- **Indicador epsilon:** distancia multiplicativa mínima para alcanzar un frente de referencia.
- **Cobertura:** proporción del espacio objetivo dominado por el frente obtenido.
- **Diversidad:** grado de distribución y uniformidad de las soluciones en el frente de Pareto.
- **Convergencia:** proximidad de las soluciones encontradas al frente de Pareto ideal.
- **Eficiencia:** al igual que en mono-objetivo, se considera el tiempo y el número de iteraciones.

Supongamos que aplicamos un algoritmo de búsqueda local para minimizar la función **Esfera** en 10 dimensiones ($f(x) = \sum_{i=1}^{10} x_i^2$), cuyo óptimo global es $f(x^*) = 0$. Tras una ejecución, el algoritmo se detiene al alcanzar el criterio de convergencia.

Paso 1. Definición de métricas de rendimiento

- **Calidad de la solución:** El valor final de $f(x)$ obtenido al final de la búsqueda.
- **Eficiencia temporal:** El tiempo total de CPU consumido.
- **Coste computacional:** El número total de iteraciones o evaluaciones de la función objetivo realizadas.

Paso 2. Simulación de resultados

Supongamos que tras ejecutar el algoritmo, obtenemos los siguientes valores de las métricas:

- Calidad (Mejor $f(x)$): 1.25×10^{-6}
- Error absoluto: $1.25 \times 10^{-6} - 0 = 1.25 \times 10^{-6}$
- Tiempo de ejecución: 0.1240 s
- Iteraciones: 1540 $\Rightarrow$ Eficiencia: 8.05×10^{-5} s/iteración.

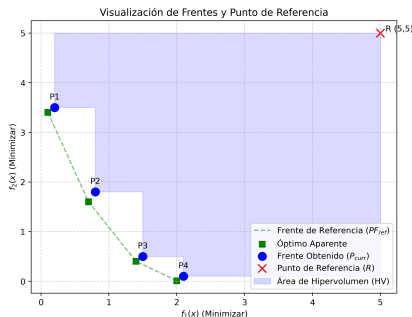
Paso 3. Interpretación de los resultados

- **Eficacia:** El algoritmo ha logrado una solución muy cercana al óptimo (error de 10^{-6}), lo que indica una alta calidad.
- **Eficiencia:** El tiempo de ejecución y el número de iteraciones deben valorarse en función de la complejidad del problema. Un trade-off equilibrado entre estas métricas es preferible.

Supongamos un problema de minimización con dos funciones objetivo. Tras una ejecución, el algoritmo devuelve un frente de Pareto aproximado (P_{curr}). Para evaluar su calidad, comparamos este conjunto con un frente de referencia (PF_{ref}) que representa el mejor conocimiento disponible hasta el momento.

Paso 1. Definición de conjuntos y puntos de referencia

- Frente Obtenido (P_{curr}): $\{(0.2, 3.5), (0.8, 1.8), (1.5, 0.5), (2.1, 0.1)\}$.
- Frente de Referencia (PF_{ref}): $\{(0.1, 3.4), (0.7, 1.6), (1.4, 0.4), (2.0, 0.01)\}$.
- Punto de Referencia (R): $(5, 5)$, necesario para calcular el hipervolumen.



Paso 2. Implementación del cálculo de métricas en Python

```
import numpy as np

# Datos de la ejecución
p_curr = np.array([[0.2, 3.5], [0.8, 1.8], [1.5, 0.5], [2.1, 0.1]])
pf_ref = np.array([[0.1, 3.4], [0.7, 1.6], [1.4, 0.4], [2.0, 0.01]])
ref_point = np.array([5.0, 5.0])

# 1. Hipervolumen (HV) en 2D
def calculate_hv_2d(front, r):
    f = front[front[:, 0].argsort()] # Ordenar por f1
    hv = (r[0] - f[0, 0]) * (r[1] - f[0, 1])
    for i in range(1, len(f)):
        hv += (r[0] - f[i, 0]) * (f[i-1, 1] - f[i, 1])
    return hv

# 2. Indicador Epsilon (mínima escala para que p_curr domine a pf_ref)
def epsilon_indicator(front, reference):
    eps_val = float('inf')
    for r in reference:
        for f in front:
            eps_val = min(eps_val, max(f / r))
    return eps_val

# 3. Diversidad (desviación estándar de distancias entre soluciones adyacentes)
def calculate_diversity(front):
    if len(front) < 2: return 0.0
    distances = np.sqrt(np.sum(np.diff(front, axis=0)**2, axis=1))
    return np.std(distances)

# 4. Convergencia (distancia media al frente de referencia)
```



```
def calculate_convergence(front, reference):
    distances = [np.min(np.linalg.norm(reference - p, axis=1)) for p in front]
    return np.mean(distances)

# Resultados (20.05)
hv_c = calculate_hv_2d(p_curr, ref_point)
hv_r = calculate_hv_2d(pf_ref, ref_point)

print(f'Hipervolumen: {hv_c:.4f}')
print(f'Cobertura (Ratio HV): {(hv_c/hv_r):.4f}')
print(f'Indicador Epsilon Multiplicativo: {epsilon_indicator(p_curr, pf_ref):.4f}')
print(f'Diversidad (Std Dev): {calculate_diversity(p_curr):.4f}')
print(f'Convergencia Media: {calculate_convergence(p_curr, pf_ref):.4f}')
```

Hipervolumen: 20.0500
Cobertura (Ratio HV): 0.9516
Indicador Epsilon Multiplicativo: 1.1429
Diversidad (Std Dev): 0.4530
Convergencia Media: 0.1602

Paso 3. Interpretación de métricas de extensión y cobertura

- **Calidad de la región dominada:** El Hipervolumen de 20.05 y la Cobertura de 0.9516 indican que el algoritmo ha capturado el 95.16% del área dominada por el frente de referencia. Es un valor muy alto, lo que demuestra que las soluciones están bien extendidas y próximas a la frontera ideal.
- **Distancia al frente óptimo:** El Indicador Epsilon de 1.1429 señala que el frente obtenido es un 14.29% peor que el de referencia en el punto más desfavorable. Para que P_{curr} igualara a PF_{ref} , sus objetivos deberían reducirse por ese factor.

Ejemplo 2: Evaluación de una ejecución multi-objetivo IV

- **Precisión del algoritmo:** La Convergencia Media de 0.1602 cuantifica la cercanía física entre los frentes. En promedio, cada solución encontrada está a solo 0.16 unidades de distancia euclídea del mejor frente conocido, confirmando una buena aproximación.
- **Distribución de soluciones:** El valor de 0.4530 en Diversidad (desviación estándar de las distancias) revela que el espaciado entre soluciones no es perfectamente uniforme. Existe cierta irregularidad en la densidad de puntos a lo largo del frente de Pareto obtenido.

Acerca del frente de referencia

- El uso de un **frente de referencia** permite una comparación objetiva incluso si no conocemos la verdadera frontera de Pareto del problema.
- En problemas reales sin óptimo conocido, el PF_{ref} se construye uniendo todas las soluciones no dominadas encontradas en todas las ejecuciones realizadas (propias y de terceros).

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - **Variabilidad en los resultados**
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

La **variabilidad en los resultados** es una característica inherente de los algoritmos estocásticos, que se debe a su dependencia de elementos aleatorios en el proceso de búsqueda. Esta variabilidad puede manifestarse de las siguientes maneras:

- **Resultados diferentes en ejecuciones independientes:**

- Los algoritmos estocásticos, como el Temple Simulado o la Búsqueda Tabú, pueden producir diferentes soluciones óptimas en ejecuciones separadas debido a la aleatoriedad en la selección de soluciones vecinas o en la aceptación de soluciones subóptimas.
- Esta variabilidad puede ser ventajosa, ya que permite explorar diversas regiones del espacio de búsqueda, pero también puede dificultar la evaluación del rendimiento del algoritmo.

- **Impacto en la calidad de la solución:**

- La calidad de las soluciones obtenidas puede variar significativamente entre ejecuciones, afectando la capacidad del algoritmo para encontrar soluciones cercanas al óptimo global.
- Es crucial evaluar la consistencia de un algoritmo estocástico mediante múltiples ejecuciones para obtener una estimación precisa de su rendimiento promedio.

- **Medición de la variabilidad:**

- Se utilizan estadísticas descriptivas, como la media y la desviación estándar en torno a una o varias métricas, para cuantificar la variabilidad en los resultados.
- Los intervalos de confianza proporcionan una medida de la precisión de las estimaciones de rendimiento, indicando el rango en el que se espera que se encuentren los resultados en futuras ejecuciones.

- **Consideraciones prácticas:**

- Al implementar algoritmos estocásticos, es importante realizar un número suficiente de ejecuciones para obtener una evaluación robusta de su rendimiento.
- La variabilidad también puede ser gestionada mediante técnicas de control de aleatoriedad, como el uso de semillas aleatorias fijas para reproducir resultados.

- La variabilidad no solo afecta la calidad de la solución, sino también el tiempo de ejecución y el número de iteraciones necesarias para converger.
- La experimentación sistemática y el análisis estadístico ayudan a comprender el comportamiento de los algoritmos estocásticos y optimizar su configuración.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - **Principios de la t de Student**
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

La prueba t de Student es una herramienta estadística utilizada para determinar si hay una diferencia significativa entre las medias de dos conjuntos de datos. En el contexto de la optimización heurística, se utiliza para comparar el rendimiento de dos algoritmos en torno a una métrica específica, como la calidad de la solución o el tiempo de ejecución. Las aplicaciones en optimización son:

- Comparar el rendimiento promedio de dos algoritmos en términos de una métrica seleccionada.
- Evaluar si las diferencias observadas en los resultados son estadísticamente significativas o si podrían ser producto del azar.

La prueba t de Student debe cumplir ciertos supuestos:

- Los datos de cada grupo **deben seguir una distribución normal**.
- Las varianzas de los dos grupos deben ser aproximadamente iguales (homogeneidad de varianzas).
- Las muestras deben ser independientes.

Hay varios tipos de prueba t:

- **Prueba t para muestras independientes:** se utiliza cuando se comparan dos grupos diferentes, como dos algoritmos distintos.
- **Prueba t para muestras relacionadas:** se aplica cuando se comparan dos conjuntos de datos relacionados, como el rendimiento del mismo algoritmo antes y después de un ajuste de parámetros.

El procedimiento para realizar la prueba t es el siguiente:

1 **Formular las hipótesis:**

- Hipótesis nula (H_0): no hay diferencia significativa entre las medias de los dos grupos.
- Hipótesis alternativa (H_a): existe una diferencia significativa entre las medias.

2 **Calcular el estadístico t:**

- Para muestras independientes:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

donde $\bar{x}_1$ y $\bar{x}_2$ son las medias de los grupos, s_1^2 y s_2^2 son las varianzas, y n_1 y n_2 son los tamaños de las muestras.

3 **Calcular los grados de libertad (df):**

- Para muestras independientes con varianzas iguales:

$$df = n_1 + n_2 - 2$$

- Para muestras independientes con varianzas desiguales (prueba t de Welch):

$$df = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{\left(\frac{s_1^2}{n_1}\right)^2}{n_1 - 1} + \frac{\left(\frac{s_2^2}{n_2}\right)^2}{n_2 - 1}}$$

- Para muestras relacionadas:

$$df = n - 1$$

donde n es el número de pares de observaciones.

4 Determinar el valor crítico y tomar una decisión:

- Comparar el valor calculado de t con el valor crítico de la distribución t para el nivel de significancia deseado (α).
 - Si $|t|$ es mayor que el valor crítico, se rechaza H_0 , indicando una diferencia significativa.
- Utilizar software estadístico o bibliotecas de programación (como SciPy en Python) para realizar la prueba t de manera eficiente.
 - Asegurarse de que los datos cumplen con los supuestos de la prueba para obtener resultados válidos.

Tabla de valores críticos de la t de Student

df	0.10	0.05	0.025	0.01	0.005	0.001
1	3.078	6.314	12.71	31.82	63.66	318.3
2	1.886	2.920	4.303	6.965	9.925	22.33
3	1.638	2.353	3.182	4.541	5.841	10.21
4	1.533	2.132	2.776	3.747	4.604	7.173
5	1.476	2.015	2.571	3.365	4.032	5.894
6	1.440	1.943	2.447	3.143	3.707	5.208
7	1.415	1.895	2.365	2.998	3.499	4.785
8	1.397	1.860	2.306	2.896	3.355	4.501
9	1.383	1.833	2.262	2.821	3.250	4.297
10	1.372	1.812	2.228	2.764	3.169	4.144
15	1.341	1.753	2.131	2.602	2.947	3.733
20	1.325	1.725	2.086	2.528	2.845	3.552
25	1.316	1.708	2.060	2.485	2.787	3.467
30	1.310	1.697	2.042	2.457	2.750	3.416
40	1.303	1.684	2.021	2.423	2.704	3.373
50	1.299	1.676	2.009	2.403	2.678	3.357
60	1.296	1.671	2.000	2.390	2.660	3.349
80	1.292	1.664	1.990	2.374	2.639	3.340
100	1.290	1.660	1.984	2.364	2.626	3.336
∞	1.282	1.645	1.960	2.326	2.576	3.291

- df: Grados de libertad.
- Columnas: Valores críticos de t para diferentes niveles de significancia (α).
- Por ejemplo, con 10 grados de libertad y un nivel de significancia de 0.05, el valor crítico de t es 1.812.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - **Estadísticas descriptivas para resumir ejecuciones**
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Las estadísticas descriptivas son herramientas fundamentales para analizar y resumir los resultados de múltiples ejecuciones de algoritmos de optimización, especialmente aquellos de naturaleza estocástica.

Media ($\bar{m}$):

- La media es el promedio de los resultados obtenidos en varias ejecuciones.
- Se calcula como $\bar{m} = \frac{1}{n} \sum_{i=1}^n m_i$, donde m_i son los resultados individuales de la métrica seleccionada y n es el número de ejecuciones.
- Proporciona una estimación del rendimiento promedio del algoritmo.

Desviación estándar (σ):

- La desviación estándar mide la dispersión de los resultados alrededor de la media.
- Se calcula como $\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (m_i - \bar{m})^2}$.
- Indica la variabilidad en el rendimiento del algoritmo; una menor σ sugiere resultados más consistentes.

Intervalos de confianza:

- Los intervalos de confianza proporcionan un rango en el que se espera que se encuentre la media verdadera del rendimiento del algoritmo con un cierto nivel de confianza.
- Para un nivel de confianza del 95%, el intervalo se calcula como $\bar{m} \pm t_{\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$, donde $t_{\alpha/2}$ es el valor crítico (bilateral) de la distribución t de Student.
- Ayudan a evaluar la precisión de la estimación de la media y a comparar la efectividad de diferentes algoritmos.

- La interpretación de estas estadísticas debe considerar el contexto del problema, la métrica seleccionada, y las características del algoritmo.
- Es esencial realizar un número suficiente de ejecuciones para obtener estimaciones fiables.
- La visualización de estos resultados mediante diagramas de caja y bigotes puede facilitar la comprensión de la variabilidad y la distribución de los datos.

Supongamos que hemos ejecutado un algoritmo de búsqueda local **30 veces** de forma independiente sobre un problema de minimización. Queremos resumir el rendimiento promedio y la fiabilidad del algoritmo basándonos en el valor final de la función objetivo.

Paso 1. Simulación de datos y cálculo de la media

- **Datos obtenidos:** Imaginemos que tras las 30 ejecuciones ($n = 30$), obtenemos un conjunto de valores cuya suma total es $\sum f_i(x) = 360$.
- **Media ($\bar{m}$):** Representa el rendimiento esperado del algoritmo.

$$\bar{m} = \frac{360}{30} = 12$$

Paso 2. Medición de la consistencia (Desviación Estándar)

- **Cálculo:** Supongamos que la dispersión de los resultados respecto a la media arroja una desviación estándar (σ) de **1.5**.
- **Interpretación:** Una σ baja indica que el algoritmo es muy **consistente**, es decir, que en casi todas las ejecuciones ofrece resultados similares a la media.

Paso 3. Cálculo del Intervalo de Confianza (95%)

Para dar una medida de la precisión de nuestra estimación, calculamos el rango donde se encuentra la "verdadera" media del algoritmo:

- **Valor crítico ($t_{\alpha/2}$):** Para $n - 1 = 29$ grados de libertad y $\alpha = 0.05$ (bilateral), consultamos la tabla y obtenemos $t_{0.025,29} \approx 2.045$.
- **Error estándar:** $\frac{\sigma}{\sqrt{N}} = \frac{1.5}{\sqrt{30}} \approx 0.274$.
- **Intervalo:** $12 \pm (2.045 \cdot 0.274) \Rightarrow [11.44, 12.56]$.

Paso 4. Interpretación y generalización

- **Conclusión:** Podemos afirmar con un **95% de confianza** que el rendimiento promedio real del algoritmo sobre este problema está entre 11.44 y 12.56.

Aplicabilidad

Este mismo análisis es aplicable a cualquier métrica cuantitativa, como el **tiempo de ejecución** o, en algoritmos multi-objetivo, el **hipervolumen** o el **indicador epsilon**.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Para evaluar y comparar el rendimiento de diferentes algoritmos de optimización en torno a una métrica, es fundamental utilizar métodos estadísticos que permitan validar la significancia de las diferencias observadas en sus resultados.

Las **pruebas de hipótesis** se utilizan para determinar si las diferencias observadas entre los resultados de dos algoritmos son estadísticamente significativas.

- **Hipótesis nula (H_0)**: no hay diferencia significativa en el rendimiento de los algoritmos.
- **Hipótesis alternativa (H_a)**: existe una diferencia significativa en el rendimiento.
- Ejemplo: prueba t de Student para comparar las medias de las métricas de dos conjuntos de resultados.

El **análisis de varianza (ANOVA)** se emplea cuando se desea comparar más de dos algoritmos simultáneamente.

- ANOVA evalúa si al menos uno de los algoritmos tiene un rendimiento significativamente diferente.
 - **Hipótesis nula (H_0)**: todas las medias de las métricas de los algoritmos son iguales.
 - **Hipótesis alternativa (H_a)**: al menos una media es diferente.
 - Si ANOVA indica diferencias significativas, se pueden realizar pruebas post-hoc (como Tukey) para identificar qué algoritmos difieren.
- Normalidad de los datos y homogeneidad de varianzas son supuestos clave para la validez de estas pruebas.
 - Si los datos no cumplen los supuestos, se pueden aplicar transformaciones o utilizar pruebas no paramétricas (como la prueba de Mann-Whitney para dos muestras o Kruskal-Wallis para más de dos).

Para comparar el rendimiento de dos algoritmos de optimización, A y B, utilizamos una prueba t de Student para muestras independientes, usando como métrica el valor de la función objetivo f .

Los resultados de las ejecuciones son:

- Algoritmo A: $f_A = \{10.2, 9.8, 10.5, 10.1, 9.9\}$
- Algoritmo B: $f_B = \{9.5, 9.7, 9.6, 9.8, 9.4\}$

Paso 1. Formular las hipótesis

- Hipótesis nula (H_0): No hay diferencia significativa en el rendimiento promedio de los algoritmos A y B.
- Hipótesis alternativa (H_a): Existe una diferencia significativa en el rendimiento promedio de los algoritmos A y B.

Paso 2. Calcular las estadísticas descriptivas

- Media de A ($\bar{x}_A$):

$$\bar{x}_A = \frac{10.2 + 9.8 + 10.5 + 10.1 + 9.9}{5} = 10.1$$

- Media de B ($\bar{x}_B$):

$$\bar{x}_B = \frac{9.5 + 9.7 + 9.6 + 9.8 + 9.4}{5} = 9.6$$

- Desviación estándar de A (σ_A):

$$\sigma_A = \sqrt{\frac{(10.2 - 10.1)^2 + (9.8 - 10.1)^2 + \dots + (10.1 - 10.1)^2 + (9.9 - 10.1)^2}{5 - 1}} \approx 0.273$$

- Desviación estándar de B (σ_B):

$$\sigma_B = \sqrt{\frac{(9.5 - 9.6)^2 + (9.7 - 9.6)^2 + (9.6 - 9.6)^2 + (9.8 - 9.6)^2 + (9.4 - 9.6)^2}{5 - 1}} \approx 0.158$$

Paso 3. Calcular el estadístico t

- Estadístico t:

$$t = \frac{\bar{x}_A - \bar{x}_B}{\sqrt{\frac{\sigma_A^2}{n_A} + \frac{\sigma_B^2}{n_B}}} = \frac{10.1 - 9.6}{\sqrt{\frac{0.273^2}{5} + \frac{0.158^2}{5}}} \approx 4.472$$

Paso 4. Determinar el valor crítico y tomar una decisión

- Grados de libertad aproximados (df):

$$df \approx \frac{\left(\frac{\sigma_A^2}{n_A} + \frac{\sigma_B^2}{n_B} \right)^2}{\frac{\left(\frac{\sigma_A^2}{n_A} \right)^2}{n_A - 1} + \frac{\left(\frac{\sigma_B^2}{n_B} \right)^2}{n_B - 1}} \approx 7.98$$

- Valor crítico para $\alpha = 0.05$ (bilateral): $t_{\alpha/2} \approx 2.306$
- Como $|t| = 4.472 > t_{\alpha/2} = 2.306$, rechazamos H_0 .

Conclusión

Existe una diferencia significativa en el rendimiento promedio de los algoritmos A y B, con un nivel de confianza del 95%.

Para comparar el rendimiento de tres algoritmos de optimización, A, B y C, utilizamos un análisis de varianza (ANOVA) de una vía.

Los resultados de las ejecuciones son:

- Algoritmo A: $f_A = \{10.2, 9.8, 10.5, 10.1, 9.9\}$
- Algoritmo B: $f_B = \{9.5, 9.7, 9.6, 9.8, 9.4\}$
- Algoritmo C: $f_C = \{10.0, 10.3, 10.1, 10.2, 10.4\}$

Paso 1. Formular las hipótesis

- Hipótesis nula (H_0): No hay diferencia significativa en el rendimiento promedio de los algoritmos A, B y C.
- Hipótesis alternativa (H_a): Al menos uno de los algoritmos tiene un rendimiento promedio diferente.

Paso 2. Calcular las estadísticas descriptivas

- Media de A ($\bar{x}_A$), B ($\bar{x}_B$), y C ($\bar{x}_C$).
- Varianza total y varianza entre grupos.

Paso 3. Realizar el ANOVA

- Calcular el estadístico F y el valor p.

Paso 4. Interpretar los resultados

- Si el valor p es menor que el nivel de significancia ($\alpha = 0.05$), rechazamos H_0 .

Código Python para realizar el ANOVA

```
import numpy as np
from scipy.stats import f_oneway

# Datos de los algoritmos
f_A = [10.2, 9.8, 10.5, 10.1, 9.9]
f_B = [9.5, 9.7, 9.6, 9.8, 9.4]
f_C = [10.0, 10.3, 10.1, 10.2, 10.4]
# Realizar ANOVA
F_statistic, p_value = f_oneway(f_A, f_B, f_C)
# Mostrar resultados
print(f'Estadístico F: {F_statistic:.2f}')
print(f'Valor p: {p_value:.4f}')
# Interpretación
if p_value < 0.05:
    print("Rechazamos la hipótesis nula => Hay diferencias significativas entre los algoritmos.")
else:
    print("No se rechaza la hipótesis nula => No hay diferencias significativas entre los algoritmos.")
```

Estadístico F: 12.40

Valor p: 0.0012

Rechazamos la hipótesis nula => Hay diferencias significativas entre los algoritmos.

Cuando el valor p del ANOVA es menor que el nivel de significancia, rechazamos la hipótesis nula (H_0) de que todas las medias son iguales. En este punto, es obligatorio realizar una prueba post-hoc (como la de Tukey) para identificar qué parejas de algoritmos presentan diferencias reales entre sí.

Paso 5. Realizar la prueba Post-hoc (Tukey HSD)

Utilizamos la prueba de Tukey para realizar comparaciones múltiples por pares y determinar dónde residen exactamente las diferencias.

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# Preparar los datos para la prueba de Tukey
data = f_A + f_B + f_C
labels = ['A'] * 5 + ['B'] * 5 + ['C'] * 5

# Realizar la prueba de Tukey HSD
tukey = pairwise_tukeyhsd(endog=data, groups=labels, alpha=0.05)
print(tukey)
```

Multiple Comparison of Means - Tukey HSD, FWER=0.05

=====						
group1	group2	meandiff	p-adj	lower	upper	reject

A	B	-0.5	0.0058	-0.8444	-0.1556	True
A	C	0.1	0.725	-0.2444	0.4444	False
B	C	0.6	0.0015	0.2556	0.9444	True

Paso 6. Interpretación Final del Análisis

Al analizar la columna reject y el meandiff (diferencia de medias), podemos concluir lo siguiente:

- **Comparación A vs B:** Se rechaza H_0 ($p < 0.05$). El Algoritmo B es significativamente mejor (menor valor) que el A.
- **Comparación A vs C:** No hay diferencias significativas ($p = 0.725$). Su rendimiento es estadísticamente equivalente.
- **Comparación B vs C:** Se rechaza H_0 ($p < 0.05$). El Algoritmo B es significativamente mejor que el C.
- **Conclusión:** El Algoritmo B es el ganador estadístico para este problema de optimización, ya que ofrece el rendimiento promedio más bajo con una confianza del 95%.

Supongamos que comparamos tres algoritmos (A, B y C) en un problema complejo donde las soluciones suelen estancarse en óptimos locales, generando distribuciones con muchos valores atípicos (outliers) que violan el supuesto de normalidad.

Paso 1. Formular las hipótesis

- **Hipótesis nula (H_0)**: Las distribuciones de los resultados de los tres algoritmos son iguales (tienen la misma mediana).
- **Hipótesis alternativa (H_a)**: Al menos uno de los algoritmos tiene una distribución de rendimiento diferente.

Paso 2. Verificación de supuestos (Shapiro-Wilk)

Antes de elegir la prueba, comprobamos la normalidad. Si el valor $p < 0.05$ en la prueba de Shapiro-Wilk, rechazamos la normalidad y debemos usar métodos no paramétricos.

```
import numpy as np
from scipy.stats import kruskal, shapiro, mannwhitneyu

# Datos simulados con distribuciones no normales (con outliers)
f_A = [10.2, 9.8, 15.5, 10.1, 9.9, 11.2, 10.0, 10.1, 14.5, 9.7]
f_B = [9.5, 9.7, 9.6, 9.8, 9.4, 9.5, 9.6, 9.9, 12.1, 9.5]
f_C = [8.1, 8.3, 8.5, 15.0, 8.2, 8.4, 8.1, 8.6, 8.2, 8.5]

# 1. Comprobación de normalidad (Ejemplo para A)
_, p_norm = shapiro(f_A)
print(f"P-valor Shapiro-Wilk (Alg. A): {p_norm:.4f}") # Suele ser < 0.05

# 2. Realizar la prueba de Kruskal-Wallis
h_statistic, p_value = kruskal(f_A, f_B, f_C)

print(f'Estadístico H: {h_statistic:.2f}')
print(f'Valor p: {p_value:.4f}')

if p_value < 0.05:
    print("Rechazamos H0 => Hay diferencias significativas (usando Kruskal-Wallis).")
```

P-valor Shapiro-Wilk (Alg. A): 0.0004
Estadístico H: 16.47
Valor p: 0.0003
Rechazamos H0 => Hay diferencias significativas (usando Kruskal-Wallis).

Paso 3. Comparaciones por pares (Post-hoc)

Si Kruskal-Wallis indica diferencias, podemos usar la prueba de **Mann-Whitney** con corrección de Bonferroni para comparar parejas (ej. A vs B, B vs C).

```
# Ejemplo: Comparación directa entre B y C
stat, p_mw = mannwhitneyu(f_B, f_C)
print(f"P-valor Mann-Whitney (B vs C): {p_mw:.4f}")
```

P-valor Mann-Whitney (B vs C): 0.0027

Paso 4. Interpretación

- Al no cumplirse la normalidad, **Kruskal-Wallis** es más robusto que ANOVA ya que se basa en el **rango** de los datos en lugar de sus valores medios.
- Si el valor $p < 0.05$, concluimos que la elección del algoritmo influye significativamente en el resultado, incluso ante la presencia de valores atípicos.
- El p-valor de 0.0027 en la prueba de **Mann-Whitney** indica que la probabilidad de que las diferencias observadas entre B y C sean producto del azar es extremadamente baja (0.27%). Por tanto, rechazamos H_0 y validamos la superioridad estadística de uno de los algoritmos sobre el otro en este problema no paramétrico.

Observando que el Algoritmo B presenta una mediana inferior y una distribución de resultados más concentrada en valores bajos, se concluye que el Algoritmo B es el más eficaz para resolver este problema bajo condiciones de no normalidad. Esta comparación se aprecia muy bien con el diagrama de cajas y bigotes, que veremos más adelante.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - **Gráficos de convergencia**
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

La visualización de resultados es esencial para interpretar y comunicar el rendimiento de los algoritmos de optimización en torno a la(s) métrica(s) seleccionadas. A continuación, se presentan dos herramientas clave para este propósito. Empezamos por los gráficos de convergencia. Los **gráficos de convergencia** muestran cómo evoluciona la función objetivo a lo largo de las iteraciones de un algoritmo.

- **Componentes:**

- El eje horizontal representa el número de iteraciones.
- El eje vertical muestra el valor de la función objetivo.
- Las curvas trazadas indican el progreso del algoritmo hacia la solución óptima.

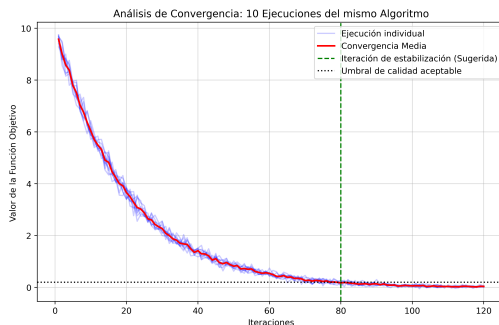
- **Interpretación:**

- Permiten evaluar la velocidad de convergencia de un algoritmo.
- Ayudan a identificar patrones de comportamiento, como estancamientos en óptimos locales o mejoras rápidas.
- Facilitan la comparación de la eficiencia de diferentes algoritmos en términos de tiempo de convergencia.

Debido a la naturaleza estocástica de los algoritmos de búsqueda local, una única ejecución no es suficiente para caracterizar su rendimiento. Al visualizar múltiples ejecuciones del mismo algoritmo sobre un problema de minimización, podemos validar su robustez y determinar el punto de "rendimientos decrecientes" para optimizar el coste computacional.

Paso 1. Recolección de datos de 10 ejecuciones independientes: Supongamos un algoritmo que busca el mínimo global ($f(x^*) = 0$). Cada ejecución tendrá una trayectoria distinta debido a la aleatoriedad en la exploración.

Paso 2. Visualización y análisis de estabilidad



Paso 3. Interpretación de los resultados

- ❶ **Convergencia al mínimo global:** Se observa que todas las ejecuciones, a pesar de sus diferencias iniciales, tienden hacia el valor óptimo cercano a 0. Esto indica que el algoritmo es **robusto** y no se queda atrapado permanentemente en óptimos locales mediocres para este problema.
- ❷ **Determinación de iteraciones máximas (max_iter):**
 - Entre la iteración 1 y la 60, hay una mejora drástica en la calidad de la solución.
 - A partir de la **iteración 80**, la pendiente de la curva media es casi nula (estancamiento), lo que sugiere que continuar la búsqueda después de este punto consume tiempo de CPU sin aportar beneficios significativos en la calidad.
 - Por tanto, se podría fijar $max_iter = 80$ para maximizar la **eficiencia temporal** sin sacrificar la eficacia.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Los **diagramas de caja y bigotes** son útiles para resumir la distribución de los resultados de múltiples ejecuciones de un algoritmo.

- **Componentes:**

- La **caja** representa el rango intercuartílico (IQR), que abarca el 50% central de los datos.
- La **línea dentro de la caja** indica la mediana, proporcionando una medida de tendencia central.
- Los **bigotes** se extienden hasta los valores más extremos que no son considerados atípicos.
- Los **puntos fuera de los bigotes** son valores atípicos, que pueden indicar ejecuciones inusuales.

- **Interpretación:**

- Permiten comparar la variabilidad y la simetría de los resultados entre diferentes algoritmos.
- Facilitan la identificación de algoritmos con resultados más consistentes y menos dispersos.

- La combinación de ambas herramientas proporciona una visión completa del rendimiento de los algoritmos, considerando tanto la calidad de las soluciones como la eficiencia del proceso de búsqueda.
- Es importante utilizar estas visualizaciones junto con análisis estadísticos para obtener conclusiones robustas sobre el rendimiento de los algoritmos.

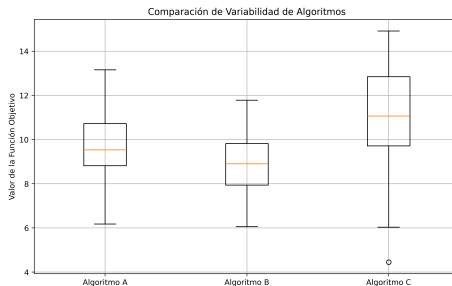
Para ilustrar cómo utilizar diagramas de caja y bigotes para comparar la variabilidad de diferentes algoritmos. Podemos simular un escenario en el que comparamos tres algoritmos de optimización: Algoritmo A, Algoritmo B y Algoritmo C. Supongamos que estos algoritmos se aplican a un problema de minimización de una función cuadrática simple, y que realizamos múltiples ejecuciones para cada algoritmo.

Paso 1. Obtención de resultados: Se recopilan los resultados de 30 ejecuciones de tres algoritmos en tres arrays `results_A`, `results_B`, y `results_C`.

Paso 2. Diagrama de caja y bigotes: Se utiliza `plt.boxplot` para crear un diagrama de caja y bigotes que compara la variabilidad de los resultados de los tres algoritmos. Este gráfico ayuda a visualizar la dispersión y los valores atípicos.

```
import matplotlib.pyplot as plt

# Gráfico de caja y bigotes
plt.figure(figsize=(10, 6))
plt.boxplot([results_A, results_B, results_C], tick_labels=['Algoritmo A', 'Algoritmo B', 'Algoritmo C'])
plt.title('Comparación de Variabilidad de Algoritmos')
plt.ylabel('Valor de la Función Objetivo')
plt.grid(True)
plt.savefig("./fig/anexo-ejemplo8.png", dpi=400, bbox_inches='tight')
```



Paso 3. Interpretación de resultados: Basándonos en la distribución de los resultados de las 30 ejecuciones, podemos extraer las siguientes conclusiones sobre el rendimiento de los algoritmos:

- **Tendencia Central (Mediana):** La línea naranja dentro de cada caja indica la mediana del valor de la función objetivo. El Algoritmo B presenta la mediana más baja, lo que sugiere que es el más eficaz para este problema de minimización.
- **Consistencia y Variabilidad (Rango Intercuartílico):** El tamaño de la caja representa el 50% central de los datos (IQR). La caja del Algoritmo B es la más estrecha, lo que indica una menor dispersión y, por tanto, una mayor consistencia en sus resultados. Por el contrario, el Algoritmo C muestra la mayor variabilidad.

Ejemplo 8: Ilustrar diagramas de caja y bigotes III

- Simetría y Robustez: Los bigotes muestran el rango de valores no atípicos. Mientras que los Algoritmos A y B presentan distribuciones relativamente simétricas, el Algoritmo C muestra una mayor dispersión hacia valores altos.
- Valores Atípicos (Outliers): Se observa un punto aislado por debajo de los bigotes en el Algoritmo C. Este representa una ejecución inusual que, aunque alcanzó un valor muy bueno, no es representativa del comportamiento habitual del algoritmo.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - **Introducción**
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Más allá de obtener valores aislados de calidad o tiempo, una evaluación completa busca diagnosticar cómo se comporta el algoritmo bajo diferentes condiciones.

Este análisis permite:

- **Identificar fortalezas y debilidades:** Detectar en qué aspectos un algoritmo supera a otros para un problema específico.
- **Fomentar la mejora continua:** Localizar áreas de mejora para impulsar nuevos enfoques.
- **Garantizar la aplicabilidad:** Asegurar que el algoritmo sea útil en entornos reales y variados.

Esta caracterización se apoya en tres pilares: la **estabilidad**, la **sensibilidad** y la **adaptabilidad**.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - **Robustez: Estabilidad y Sensibilidad**
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

La estabilidad evalúa la capacidad del algoritmo para producir resultados consistentes bajo condiciones similares.

- **Fundamento estadístico:** Se basa en la interpretación de la desviación estándar (σ) y los intervalos de confianza obtenidos en múltiples ejecuciones.
- **Interpretación:**
 - Una σ baja indica que el algoritmo es muy **consistente** y fiable.
 - Una variabilidad alta sugiere que el algoritmo es excesivamente sensible a los componentes aleatorios de la búsqueda.
- **Trade-off:** A menudo existe un compromiso entre robustez y eficiencia; un algoritmo muy estable puede requerir más tiempo de ejecución para garantizar soluciones de alta calidad.

El análisis de sensibilidad examina cómo las variaciones en los parámetros de entrada (como la temperatura inicial en Temple Simulado o la tasa de mutación en un GA) afectan al rendimiento.

- **Objetivo:** Identificar los **parámetros críticos** que influyen significativamente en el éxito del algoritmo.
- **Método:** Variar sistemáticamente un parámetro manteniendo constantes los demás para observar el efecto en la función objetivo.
- **Utilidad:** Permite ajustar la configuración del algoritmo para optimizar su convergencia y evitar una exploración insuficiente o un gasto computacional innecesario.

El objetivo del **análisis de sensibilidad** es evaluar cómo las variaciones en el parámetro de temperatura inicial (T_0) afectan el rendimiento del algoritmo de Temple Simulado. Para obtener resultados estadísticamente sólidos, realizaremos **30 ejecuciones independientes** por cada valor de T_0 , evaluando tanto la calidad media como la consistencia (estabilidad).

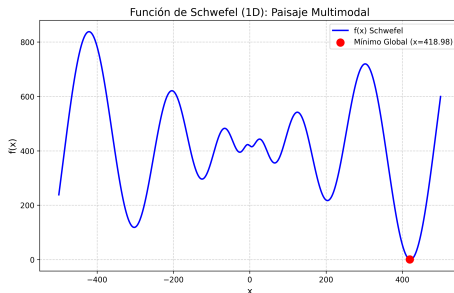
Paso 1. Definir el problema y el algoritmo

- **Función objetivo:** Schwefel (multimodal, con muchos óptimos locales).

$$f(x) = 418.9829 - x \cdot \sin(\sqrt{|x|})$$

- **Rango de búsqueda:** $x \in [-500, 500]$.

- **Métricas:** Media ($\bar{m}$) del valor final de y desviación estándar (σ).



Paso 2. Implementar el análisis robusto en Python

```
import numpy as np
# Parámetros experimentales
bounds = (-500, 500)
T0_values = [1, 10, 50, 100, 500]
runs_per_T0 = 30
results_summary = []

for T0 in T0_values:
    # 30 ejecuciones por cada configuración para garantizar rigor
    run_data = [simulated_annealing(schwefel, bounds, T0, 0.95, 2000)
                 for _ in range(runs_per_T0)]
    results_summary.append((T0, np.mean(run_data), np.std(run_data)))

print(f"{'T0':>5} | {'Media f(x)':>12} | {'Desv. Est. ':>15}")
print("-" * 40)
for t, m, s in results_summary:
    print(f"{t:5d} | {m:12.4f} | {s:15.4f}")
```

T0	Media $f(x)$	Desv. Est.
1	200.1767	130.4169
10	160.6630	121.9543
50	182.3436	129.0792
100	131.7053	118.3576
500	173.7719	132.6945

Paso 3. Interpretación de los resultados

- 1 **Identificación del parámetro crítico:** El análisis muestra que el rendimiento es altamente sensible a T_0 . En este caso, $T_0 = 100$ ofrece el mejor equilibrio entre exploración y explotación, logrando la media y la desviación más baja.
- 2 **Análisis de Estabilidad:** La σ no es especialmente baja lo que indica que el algoritmo **no es estable**. Esto se debe a que la función de Schwefel es especialmente compleja.
- 3 **Validación Estadística:** Aunque $T_0 = 100$ parece mejor que $T_0 = 500$, para afirmar que existe una "superioridad estadística" deberíamos aplicar una prueba t de Student o un ANOVA sobre los 30 datos de cada grupo.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - **Adaptabilidad**
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

La adaptabilidad evalúa la capacidad del algoritmo para ajustarse a diferentes tipos de problemas y condiciones de ejecución.

Un algoritmo se considera adaptable si mantiene un **buen rendimiento promedio** y **consistencia** en una variedad de escenarios:

- **Problemas Unimodales:** Capacidad para converger rápidamente al óptimo global cuando no hay distracciones (ej. función Esfera).
- **Problemas Multimodales:** Capacidad para explorar el espacio de búsqueda y escapar de múltiples óptimos locales (ej. función Rastrigin).

La experimentación en bancos de pruebas con características distintas determina si un algoritmo es versátil o si su eficacia está limitada a un nicho de problemas específico.

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Esta sección establece los pasos sistemáticos para evaluar un algoritmo o comparar varios entre sí, garantizando rigor estadístico y validez en los resultados.

1 Fase 1: Diseño experimental y ejecución

- **Definir el escenario:** Seleccionar problemas de prueba (benchmarks) representativos, tanto sencillos (unimodales) como complejos (multimodales).
- **Control de estocasticidad:** Debido a la naturaleza aleatoria de las heurísticas, realizar al menos 30 ejecuciones independientes por cada configuración.
- **Reproducibilidad:** Fijar semillas aleatorias y documentar todos los parámetros de control (temperatura, tasas de mutación, etc.).

2 Fase 2: Recolección de métricas de rendimiento **Para algoritmos mono-objetivo:**

- **Calidad:** Valor final de la función objetivo $f(x)$.
- **Eficiencia:** Tiempo de CPU consumido y número de iteraciones o evaluaciones.

Para algoritmos multi-objetivo:

- **Calidad y Extensión:** Hipervolumen (HV) respecto a un punto de referencia.
- **Proximidad:** Indicador Epsilon (ϵ) respecto a un frente de referencia.
- **Distribución:** Índices de diversidad y cobertura del frente de Pareto.

3 Fase 3: Análisis estadístico y validación

- **Cálculo descriptivo:** Obtener la media ($\bar{m}$) para el rendimiento esperado y la desviación estándar (σ) para la consistencia.
- **Estimación de precisión:** Calcular intervalos de confianza (usualmente al 95%) para situar la media real del algoritmo.
- **Contraste de hipótesis (Comparación):**
 - Si se comparan 2 algoritmos: Aplicar la prueba t de Student.
 - Si se comparan 3 o más: Realizar un ANOVA seguido de una prueba post-hoc (Tukey) para identificar diferencias por pares.
 - Si los datos no son normales: Utilizar pruebas no paramétricas como Kruskal-Wallis o Mann-Whitney.

4 Fase 4: Análisis visual y diagnóstico

- **Curvas de convergencia:** Graficar la evolución de la métrica frente a las iteraciones para identificar estancamientos y ajustar criterios de parada.
- **Diagramas de caja y bigotes (Boxplots):** Visualizar la dispersión, simetría y detectar valores atípicos (outliers) en los resultados.
- **Análisis de frentes (Multi-objetivo):** Representar los frentes de Pareto obtenidos frente al de referencia para evaluar visualmente la convergencia y diversidad.

5 Fase 5: Caracterización final del algoritmo

- **Diagnóstico de robustez:** Interpretar la estabilidad a partir de la variabilidad observada (σ).
- **Sensibilidad:** Determinar qué parámetros son críticos mediante variaciones sistemáticas de su valor.
- **Adaptabilidad:** Concluir si el algoritmo mantiene su eficacia al cambiar de tipo de problema (unimodal vs. multimodal).

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

- José L. Risco-Martín
- Eva Besada Portas
- Rafael del Vado Vírveda

- 1 **Introducción**
 - Evaluación y comparación de algoritmos
- 2 **Métricas de rendimiento**
 - Métricas para algoritmos mono-objetivo
 - Métricas para algoritmos multi-objetivo
 - Resumen de métricas
- 3 **Validación estadística**
 - Variabilidad en los resultados
 - Principios de la t de Student
 - Estadísticas descriptivas para resumir ejecuciones
 - Métodos estadísticos para comparar ejecuciones
- 4 **Análisis visual de resultados**
 - Gráficos de convergencia
 - Diagramas de caja y bigotes
- 5 **Caracterización del algoritmo**
 - Introducción
 - Robustez: Estabilidad y Sensibilidad
 - Adaptabilidad
- 6 **Guía práctica de evaluación y comparación**
 - Fases de evaluación y comparación
- 7 **Autores y licencia**
 - Autores
 - Licencia

Este trabajo se publica bajo una licencia **CC BY-SA 4.0**. Usted es libre de:

- **Compartir**: copiar y redistribuir el material en cualquier medio o formato.
- **Adaptar**: remezclar, transformar y construir a partir del material para cualquier propósito, incluso comercialmente.

Bajo los términos **atribución** y **compartir igual** que establece la licencia.

